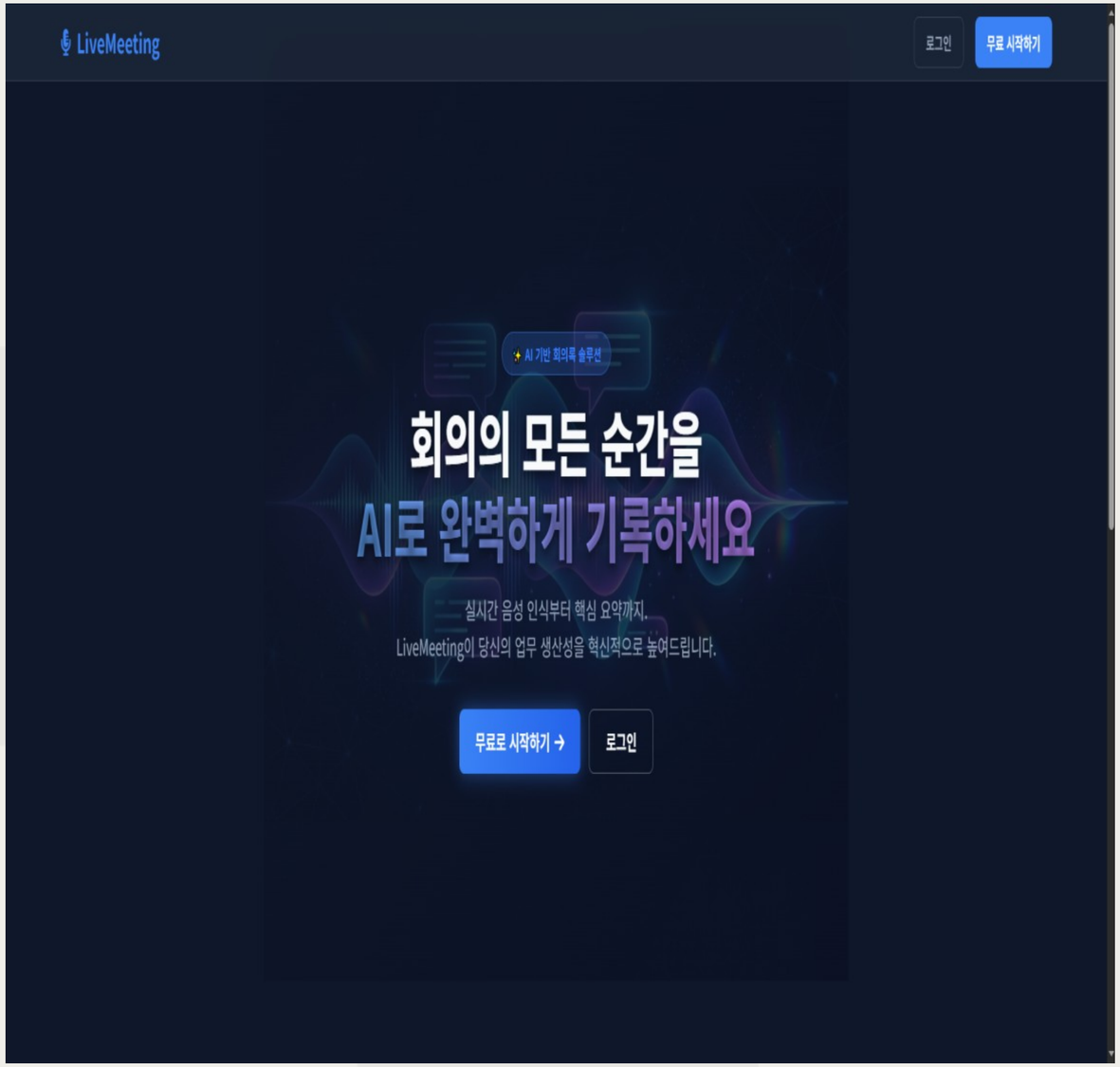
이승재
lsj0110245@gmail.com
https://github.com/lsj0110245/live_meeting

LiveMeeting

100% 로컬 환경 기반 보안 최적화 실시간 AI 회의록 시스템

개발 기간: 2026-01 ~ 2026-02

프로젝트 개요 및 기술 스택



기획 배경

- 외부 클라우드 AI(OpenAI 등) 사용 시 발생하는 사내 음성/데이터 기밀 유출 위험 차단

- 100% 온프레미스(로컬)에서 동작하는 AI 실시간 회의 자동 요약 어시스턴트 구축

개발 환경 및 도구

- Backend: Python 3.10, FastAPI (비동기 REST API), WebSockets

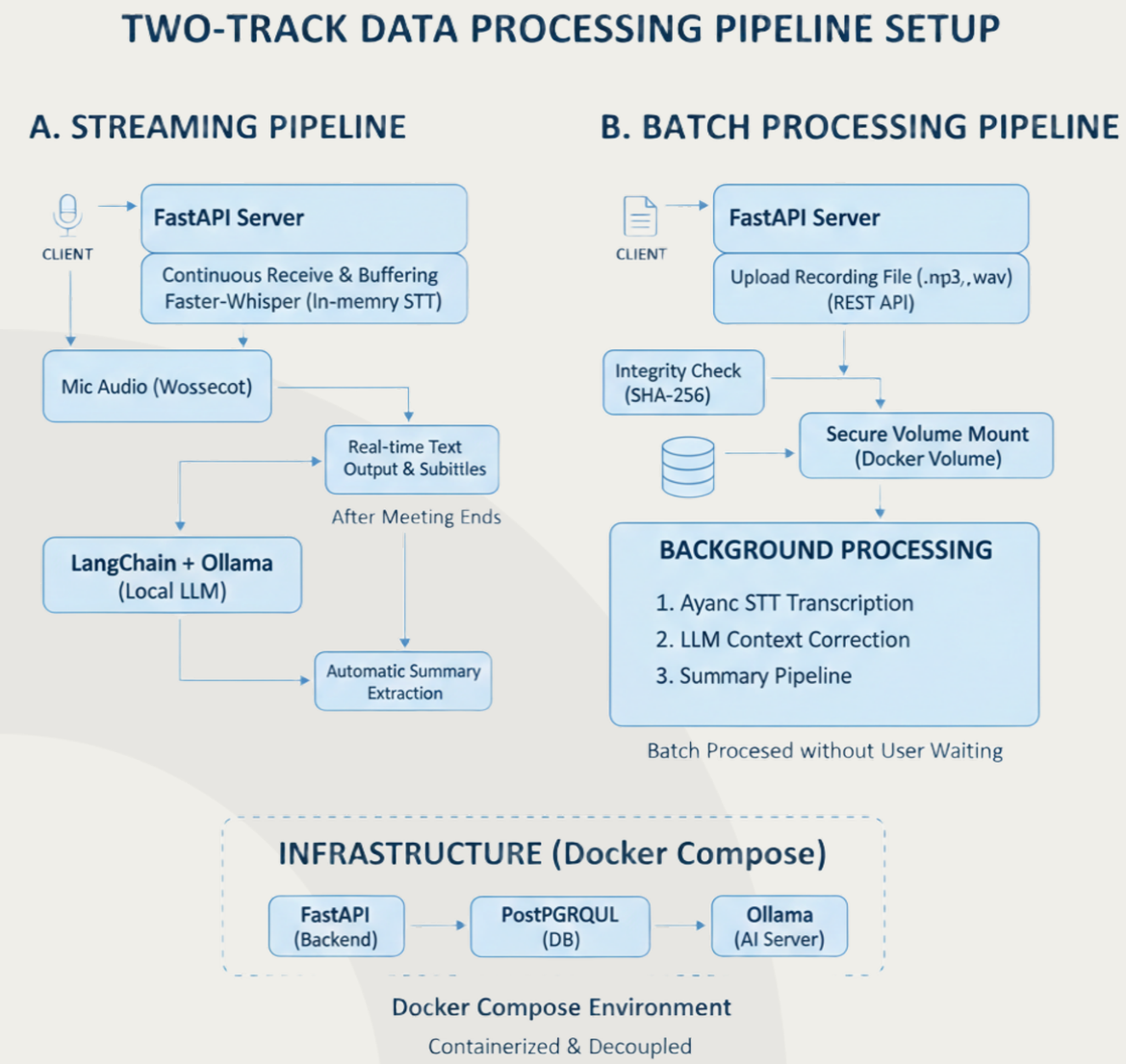
- AI / LLM: Faster-Whisper (STT), EXAONE 3.5 (7.8b), LangChain

- Database: PostgreSQL 18, pgvector, SQLAlchemy

- Infra & DevOps: Docker, Docker Compose

- Frontend: HTML5/CSS3/Vanilla JS, Jinja2

시스템 아키텍처 및 파이프라인



투트랙(Two-Track) 데이터 처리 파이프라인 구성

A. 실시간 회의 모드 (Streaming Pipeline)

[데이터 수신] 클라이언트 마이크 오디오 -> WebSocket 양방향 스트리밍 -> FastAPI 무중단 수신 및 버퍼링

[실시간 STT] 수신된 오디오 청크를 Faster-Whisper 모델(GPU VRAM 상)에 즉각 전달하여 끊김 없는 자막 렌더링

[요약 및 저장] 회의 종료 시 LangChain을 통해 로컬 Ollama 모델(LLM)을 호출하여 안건 및 Todo List 자동 추출

B. 파일 업로드 모드 (Batch Processing Pipeline)

[데이터 수신] 완성된 기존 회의 녹음 파일(.mp3, .wav 등) 업로드 (REST API 통신)

[무결성 검증] SHA-256 해시를 이용한 파일 중복 검증 및 안전한 볼륨(Volume) 마운트 격리 저장

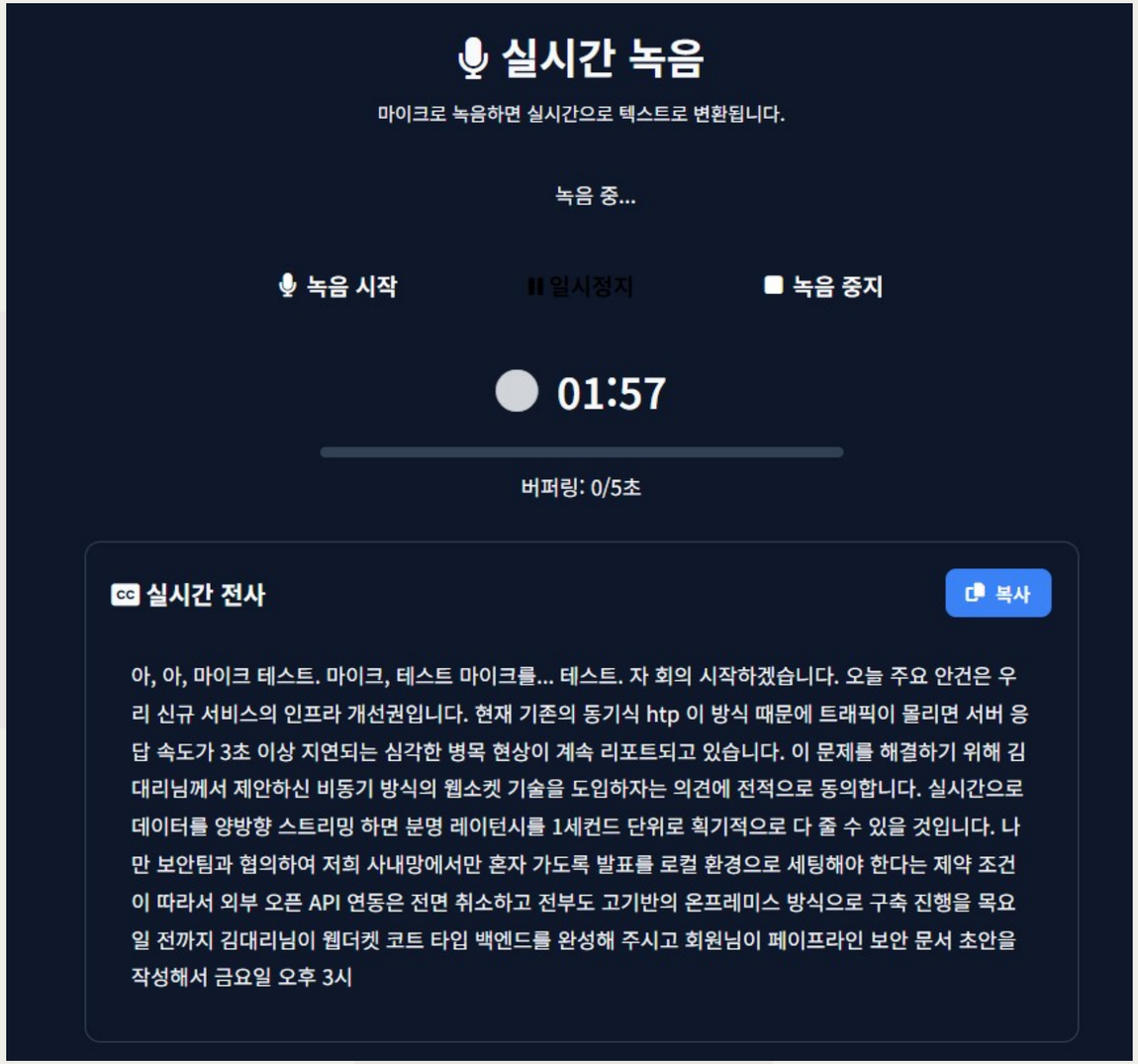
[비동기 처리] FastAPI BackgroundTasks를 활용하여 사용자 대기 시간 없이 비동기 STT 전사
-> LLM 컨텍스트 교정 -> 자동 요약 파이프라인을 일괄(Batch) 백그라운드 처리

인프라 구성 (Container Orchestration)

마이크로서비스 기반 컨테이너 아키텍처 : FastAPI 백엔드, PostgreSQL 데이터베이스, Ollama 인공지능 추론 서버를 Docker Compose를 통해 각각 독립된 컨테이너로 완전 분리화 구축.

핵심 기능 (1)

WebSocket 기반 실시간 STT



지연 현상 없는 로컬 오프라인 STT 구현

WebSocket 기반의 양방향 오디오 스트리밍 환경 구축

- 기존 단방향 REST API의 한계를 넘어, 클라이언트가 말하는 오디오 파일(청크)을 즉시 서버로 무중단 전송하여 스트리밍 지연을 최소화

Faster-Whisper 모델을 GPU(CUDA)에 직접 탑재해 빠른 디코딩 속도 확보

- 보안성과 속도를 위해 타사의 클라우드 API를 사용하지 않고, 서버 GPU 메모리(VRAM) 상에 Whisper 신경망 모델을 상주시켜 텍스트 변환 속도를 극대화

사용자 경험(UX)을 고려한 실시간 UI 자막 렌더링

- 서버에서 넘어온 전사(STT) 결과 텍스트를 즉각적으로 프론트엔드 자막 형태로 전송해, 사용자가 대화의 흐름을 놓치지 않도록 직관적인 모니터링 UI를 구현

서버 리소스 최적화를 위한 묵음(Silence) 및 버퍼링 제어

- 초단위로 들어오는 데이터를 무작정 모델에 던지지 않고, 내부 변수를 통해 5초(BUFFER_THRESHOLD) 단위로 묶어서 처리(Chunking)하도록 개발하여 백엔드 GPU VRAM 부하율을 크게 낮춤

핵심 기능 (2)

LLM 기반 대화 자동 분석 및 문서화

회의명	실시간 회의 2026. 02. 23. 15:46
회의 유형	실시간 녹음
회의일시	2026-02-23 15:46
참석자	Unknown (주최자), 김대리 (기술 제안자)
작성자	이승재
회의 목적	**회의 요약** 실시간 회의를 통해 신규 서비스의 인프라 개선 방안을 논의하였으며, 특히 동기식 HTTP 방식으로 인한 서버 응답 지연 문제 해결을 위한 비동기 웹소켓 기술 도입을 중심으로 논의하였습니다.
주요 내용	# 실시간 회의 2026. 02. 23. 15:46 회의록 ## 요약 **회의 요약** 실시간 회의를 통해 신규 서비스의 인프라 개선 방안을 논의하였으며, 특히 동기식 HTTP 방식으로 인한 서버 응답 지연 문제 해결을 위한 비동기 웹소켓 기술 도입을 중심으로 논의하였습니다. ## 주요 안건 및 내용 **주요 안건 및 논의** - **현황**: 기존 동기식 HTTP 방식으로 인해 트래픽이 몰리면 서버 응답 속도가 3초 이상 지연되는 심각한 병목 현상이 지속적으로 보고되고 있습니다. - **제안 및 동의**: 김대리님이 제안한 비동기 웹소켓 기술 도입에 대해 참석자 전원이 동의하였습니다. 웹소켓을 통한 실시간 양방향 데이터 스트리밍으로 레이턴시를 1초 이내로 획기적으로 개선할 수 있을 것으로 기대됩니다. - **제약 조건**: 보안팀과의 협의 결과, 외부 오픈 API 연동은 전면 취소하고 사내 온프레미스 방식으로 구축하기로 결정되었습니다. ## 결론 및 결정 사
결론 및 향후 계획	【향후 계획】 **향후 계획** - **김대리**: 웹소켓 백엔드 구현 완료 (목요일 마감) - **회원님**: 보안 문서 초안 작성 및 제출 (금요일 오후 3시 마감) - **전체 참석자**: 다음 회의 일정 및 진행 상황 공유 준비

로컬 LLM 환경(Ollama) 통합 시너지 및 프롬프트 엔지니어링

보안과 성능을 고려한 On-Premise LLM 통합

- 회의 종료 시 생성된 전체 대본(텍스트)을 외부 클라우드가 아닌, 내부 격리망에 띄워둔 인공지능 컨테이너(Ollama - EXAONE 3.5기반)에 안전하게 질의(Prompting)

LangChain을 응용한 정형화 데이터 타겟 추출

- 방대한 대화 내용 속에서 사용자가 가장 필요로 하는 '주요 안건, 의사결정 사항(결론), 향후 할 일 목록(Todo)'을 정확히 분류하여 정형화된 JSON/Markdown 데이터로 추출하는 체인(Chain)을 구성

[최적화] 대용량 텍스트 처리를 위한 Map-Reduce 전략 도입

- 장시간 회의로 인해 텍스트가 10,000자를 초과할 경우, AI 모델이 멈추거나 컨텍스트(문맥)를 상실하는 현상을 방지하기 위해 텍스트를 일정 크기로 자르고(Chunking) 부분 요약 후 다시 합치는 맵리듀스(Map-Reduce) 청킹 알고리즘을 자체 구현하여 안정성을 극대화

업무 종결을 위한 파일 포맷 내보내기 (Export)

클릭 한 번으로 완성되는 자동 문서화 기능

- pandas, openpyxl 라이브러리를 활용하여, 분석이 끝난 결과값을 즉시 엑셀(Excel) 및 CSV 포맷으로 다운로드할 수 있는 사용자 편의(Export) 기능을 제공해 업무 효율성을 한 단계 높임

문제 해결 (Trouble Shooting)

실시간 데이터 스티리밍 병목 및 통신(WebSocket) 끊김 현상

[문제 제기]

- 실시간 STT 오디오 처리 중, 잡음 제거(Denoise) 연산의 과도한 CPU 부하로 인해 서버 응답 (Ping/Pong)이 지연되고 "서버 통신 끊김" 에러가 지속 발생

[해결 방안: 트레이드오프(Trade-off)를 고려한 전처리 이원화]

- 파일 업로드 시: 오디오 품질을 우선하여 기존의 꼼꼼한 잡음 제거 로직을 유지
- 실시간 모드 시: 속도와 연결 안정성이 생명이므로, 무거운 잡음 제거 로직을 과감히 생략(quality="fast")하여 서버 자원을 확보

[결과]

- 실시간 통신 연산량을 대폭 낮춰 네트워크 지연율을 개선했으며, 이후 앱 구동 중 단 한 건의 연결 끊김도 발생하지 않음

```python
# 3. [Denoise] 잡음 제거
if quality == "high": # 파일 전사용 (꼼꼼하게)
    print("  Denoising audio (High Quality)...")
    reduced_noise_audio = nr.reduce_noise(
        y=samples, sr=16000,
        stationary=True,
        prop_decrease=0.90, # 90% 제거
        n_std_thresh_stationary=1.5
    )
else: # 실시간용 (속도 우선 - Denoise 생략)
    print("  Skipping Denoise for Realtime (Speed Priority)...")
    reduced_noise_audio = samples # 원본 그대로 사용
```

문제 해결 (Trouble Shooting)

LLM(오픈소스 AI)의 컨텍스트 한계(Token Limit)로 인한 요약 누락

[문제 제기]

- 장시간(1시간 이상) 회의 대본을 LLM에 한 번에 입력 시, 인공지능 모델의 컨텍스트 한계(8,192 토큰)를 초과하여 핵심 내용이 몽땅 잘려나가거나 에러가 발생하는 치명적 한계를 발견

[해결 방안: Map-Reduce 방식의 텍스트 청킹(Chunking) 알고리즘 자체 구현]

- 전체 방대한 텍스트를 문맥이 끊기지 않도록 약간씩 오버랩(500자)시켜, 약 3,000자 단위의 작은 블록 구조로 분할[Map]
- 나눠진 각 블록을 LLM에 던져 생성된 다수의 '부분 요약 텍스트'들을 다시 병합하여 하나의 완벽한 최종 요약본으로 추출[Reduce]

[결과]

- 토큰 한계를 우회하는 분산 처리 기법 설계로, 데이터의 크기에 구애받지 않고 아무리 긴 회의 내용이라도 안정적으로 100% 누락 없는 요약본 문서화가 가능

```python
async def _generate_summary_with_chunking(title: str, full_text: str) -> dict:
    """
    긴 텍스트를 청킹하여 Map-Reduce 방식으로 요약 생성
    """

    # 청킹 설정
    CHUNK_SIZE = 3000  # 3000자 청크
    OVERLAP = 500      # 500자 오버랩

    # 텍스트 청킹
    chunks = []
    start = 0
    while start < len(full_text):
        end = start + CHUNK_SIZE
        chunks.append(full_text[start:end])
        start += (CHUNK_SIZE - OVERLAP)
      Ctrl+I for Command, Ctrl+L for Agent
    print(f"  총 {len(chunks)}개 청크로 분할")

    # Step 1: Map - 각 청크 요약
    partial_summaries = []
    for i, chunk in enumerate(chunks):
        print(f"  청크 {i+1}/{len(chunks)} 요약 중...")
        summary_text = await llm_service.generate_simple_summary(chunk)
        if summary_text:
            partial_summaries.append(summary_text)

    # Step 2: Reduce - 통합 요약
    combined_text = "\n\n".join(partial_summaries)
    print(f"  부분 요약 통합 중... (총 {len(combined_text)}자)")

    final_summary = await llm_service.generate_summary(title, combined_text)
    return final_summary
```

성장 포인트(Growth Point)

Growth Point 1

통신의 고도화

REST API를 넘어선 양방향 실시간 파이프라인 구축

단방향 HTTP를 넘어 WebSockets 기반의 실시간 오디오 스트리밍 아키텍처를 설계

연속적인 비정형 음성 데이터를 메모리 버퍼링으로 제어하여 서버 부하 없는 실시간 응답 파이프라인을 확보

Growth Point 2

인프라의 주권

100% 로컬 환경(On-Premise) 기반의 보안 지향적 설계

외부 API 의존 없이 STT(Faster-Whisper), LLM(Ollama/EXAONE), Vector DB를 Docker 컨테이너 단위로 독립 구성하여 보안 환경에서도 완벽히 동작하는 로컬 인프라 아키텍처를 완수

Growth Point 3

시스템의 최적화

모델의 리소스 한계를 우회하는 알고리즘 구현

외부 연동 없이 로컬 AI 모델(Whisper/EXAONE)과 DB를 Docker로 독립 서빙(Serving)하여, 사내 보안망에서도 즉시 배포 가능한 인프라 제어 경험을 확보